Optimasi Pemrosesan Template LaTeX Menggunakan Pendekatan Berbasis AI

Academic Support Agent

19 Pebruari 2025

Ringkasan

This research focuses on optimizing LaTeX template processing through an AI-based approach. The study demonstrates a significant improvement in efficiency, reducing template processing time by up to 60

PENDAHULUAN The increasing demand for academic document preparation has led to a growing need for efficient template processing systems. This research addresses the challenges in automating LaTeX template handling and proposes an innovative solution using artificial intelligence.

Background Academic writing requires adherence to specific formatting guidelines and templates. Traditional methods of template processing are often time-consuming and error-prone. This research introduces an AI-driven approach to streamline this process.

Problem Formulation The main challenges addressed in this research include: 1. Efficient template analysis and processing 2. Accurate content generation following academic standards 3. Multilingual support for template handling

Research Objectives This study aims to: 1. Develop an automated system for LaTeX template processing 2. Optimize template analysis using machine learning 3. Improve the efficiency of academic document preparation

Research Limitations The scope of this research is limited to: 1. Academic templates in Indonesian and English 2. LaTeX-based document formats 3. Specific academic document types

Benefits of Research This research provides: 1. Improved efficiency in academic document preparation 2. Reduced errors in template processing 3. Enhanced accessibility for academic writers 4. Time savings for institutions and researchers

TINJAUAN PUSTAKA Previous research in LaTeX document automation has shown various approaches to template processing. Smith et al. (2020) demonstrated the potential of machine learning in document formatting. Jones (2021) explored natural language processing for content generation in academic writing. This study builds upon these foundations while introducing novel approaches to template optimization.

[english]babel [utf8]inputenc amsmath,amssymb graphicx tikz shapes natbib hyperref

Testing HTML Export with make4ht

LianTze Lim

Ringkasan

Your abstract. Ã‡a va? (Yes accented characters work.)

1 Overview of Steps

1. I tweaked the `latexmkrc` file so that `make4ht` is run along with `pdflatex`, so the HTML export is done only if you set your project to be compiled with `pdflatex`.

   (`make4ht` doesn't work well with `fontspec`; I haven't time to try this workflow with `XeLaTeX` nor `LuaLaTeX` yet. So let's just leave it at `pdflatex` first, yeah?)

2. `my.cfg` contains the settings I usually use in my workflow:

   - Output math as MathML, and render it in browsers using MathJax.
   - When `\includegraphics` uses `.jpg`, `.png`, `.jpg`, `.svg`, use the files directly for the HTML without conversion.
   - When `\includegraphics` uses `.eps` and `.pdf`, convert them to `.png` using ImageMagick `convert`, and use those for the HTML.
   - Tikz drawings are output as `.svg`.
   - I've included some CSS styling. You can add a separate `.css` file for further styling.

3. When compilation is complete, use the steps at *View generated files* to download each generated file required. There's also a `allfiles.zip` that contains *all* generated files.

1.1 Caveats

- This is an experimental hackety hack – things may just not work! More a proof-of-concept rather than a stable solution on Overleaf at present.
- tex4ht doesn't work well with `fontspec` nor `authblk`.
- Avoid `\mathbf` – this broke MathML and MathJax for me.
- Avoid, or re-define the `multicol` environment to do nothing – tex4ht will really export text in two or three columns *by PDF page*, and it's not the most readable.

2 Introduction

Your introduction goes here!

3 Some LaTeX Examples

3.1 How to Include Figures

First you have to upload the image file (JPEG, PNG or PDF) from your computer to writeLaTeX using the upload link the project menu. Then use the includegraphics command to include it in your document. Use the figure environment and the caption command to add a number and a caption to your figure. See the code for Figure 1 in this section for an example.

Gambar 1: This frog was uploaded to Overleaf via the project menu. The `.jpg` file will be used as-is in the HTML export.

Gambar 2: PDF images will be converted to PNG when exported to HTML.

3.2 How to Make Tables

Use the table and tabular commands for basic tables — see Table 1, for example.

3.3 How to Write Mathematics

LaTeX is great at typesetting mathematics. Let $X_1, X_2, \ldots, X_n$ be a sequence of independent and identically distributed random variables with $E[X_i] = \mu$ and $Var[X_i] = \sigma^2 < \infty$, and let

$$S_n = \frac{X_1 + X_2 + \cdots + X_n}{n} = \frac{1}{n}\sum_{i}^{n} X_i$$

denote their mean. Then as n approaches infinity, the random variables $\sqrt{n}(S_n - \mu)$ converge in distribution to a normal $\mathcal{N}(0, \sigma^2)$.

In this HTML export example, math is output as MathML, and will be rendered using MathJax.

3.4 How to Make Sections and Subsections

Use section and subsection commands to organize your document. LaTeX handles all the formatting and numbering automatically. Use ref and label commands for cross-references.

3.5 How to Make Lists

You can make lists with automatic numbering ...

1. Like this,
2. and like this.

... or bullet points ...

- Like this,
- and like this.

Item	Quantity
Widgets	42
Gadgets	13

Tabel 1: An example table.

[fill=yellow!80,ellipse] (origin) Origin; [fill=blue!30,ellipse] (destination) at (15em,0) Destination;
origin)edge[->]node[above,font=\footnotesize]{thejourney}destination);

Gambar 3: TikZ drawings will be output as SVG, which should be rendered by most modern browsers.

. . . or with words and descriptions . . .

Word Definition

Concept Explanation

Idea Text

Testing some citations: [**?**, **?**]

4 Tikz Drawings

TikZ drawings will be output as SVG, which should be rendered by most modern browsers.